API REST CON NODE.JS Y MONGODB

COOPERATIVA UNIVERSITARIA DE AHORRO Y CRÉDITO EDUCATIVO

CORPORACIÒN UNIVERSITARIA AUTONOMA DEL CAUCA

DIEGO FERNANDO PRADO OSORIO

CRISTIAN DAVID TOMBE YALANDA
LAURA SOFIA SALAZARA SANDOVAL

BASES DE DATOS AVANZADAS

POPAYÁN CAUCA
2026

Índice

1. Introducción .....
2. Objetivos .....
3. Alcance del MVP .....
4. Tecnologías utilizadas .....
5. Arquitectura del sistema .....
6. Historias de usuario .....
7. Diseño de la base de datos .....
8. Endpoints de la API REST .....
9. Reglas de negocio .....
10. Pruebas funcionales con Postman .....
8.1 Preparación.....
8.2 Pruebas funcionales (10 funcionalidades) .....
11. Conclusiones .....

1. Introducción

La **Cooperativa Universitaria de Ahorro y Crédito Educativo** es un fondo de la comunidad universitaria. Los estudiantes depositan sus ahorros, pueden solicitar préstamos destinados a gastos educativos con un interés fijo del 5 % y, a su vez, reciben una parte proporcional de los intereses generados según lo que tengan ahorrado. El Rector necesita estadísticas para supervisar el fondo.

El problema que resuelve el proyecto es llevar estas operaciones (registro de personas, ahorros, préstamos, pagos e intereses) de forma ordenada, consistente y consultable, en lugar de manejarlas manualmente. Para ello se construyó una **API REST con Node.js, Express y MongoDB** que implementa las diez funcionalidades (HU1 a HU10) derivadas del proyecto de MongoDB de la cooperativa, y que se evalúa como MVP en el parcial de “API REST con Node.js y MongoDB”.

1.1 Cómo se elaboró este documento

Se analizaron los tres archivos entregados: el PDF con los requisitos oficiales, el ZIP con el proyecto `cooperativa-api` y el TXT con el diseño y las consultas originales de MongoDB. El código no fue modificado. Todo lo que aquí se describe proviene de esos archivos; lo que no pudo comprobarse se marca como **Por verificar**.

1.2 Qué se verificó y qué no

En el entorno de revisión no había un servidor MongoDB disponible, por lo que **no se pudo ejecutar la API completa**. Se distingue así entre revisión estática y prueba real:

Verificación	Resultado	Tipo
Sintaxis de los 33 archivos `.js` de `src/` y `database/` (`node --check`)	Sin errores de sintaxis	Prueba real
`npm install` con el `package.json` y el `package-lock.json` entregados (Node v22.22.2)	Dependencias instaladas sin errores	Prueba real
13 peticiones HTTP contra la aplicación Express **sin MongoDB** (validaciones, 404, JSON inválido y `/api/health`)	Respuestas JSON y códigos coherentes; la API no se detuvo (ver sección 13.1)	Prueba real parcial
Conexión a MongoDB, `npm run seed`, persistencia de datos y respuestas exitosas (200/201) de las 10 funcionalidades	No ejecutado: requiere MongoDB	Por verificar
Lectura del código de rutas, controladores, servicios, validaciones y de la colección Postman (53 peticiones, todas con scripts de prueba)	Coherente con las historias de usuario	Revisión estática

2. Objetivos

2.1 Objetivo general

Desarrollar, documentar y sustentar un MVP de API REST con Node.js, Express y MongoDB que permita gestionar los ahorros, los préstamos educativos, los pagos y la distribución proporcional de intereses de la Cooperativa Universitaria de Ahorro y Crédito Educativo.

2.2 Objetivos específicos

- Registrar estudiantes y funcionarios y administrar cuentas de ahorro con depósitos y retiros validados (HU1 a HU4).
- Gestionar préstamos educativos: solicitud con validaciones, cálculo del interés del 5 %, consulta de saldos pendientes y pagos a capital o interés (HU5 a HU8).
- Distribuir los intereses generados de forma proporcional entre los ahorradores y ofrecer estadísticas al Rector (HU9 y HU10).
- Persistir todos los datos en MongoDB, con variables de entorno para la conexión y sin credenciales en el código.
- Responder siempre en JSON con códigos HTTP coherentes y manejar errores sin detener la aplicación.
- Documentar las historias de usuario, probar cada funcionalidad con Postman y verificar la persistencia en MongoDB Compass.

3. Alcance del MVP

El MVP abarca el ciclo completo de la cooperativa: personas → ahorros → préstamos → pagos → intereses → reportes. La siguiente tabla separa lo que **está implementado en el código** de su estado de verificación.

Funcionalidad	Implementada en el código	Probada con MongoDB
HU1 Registrar y consultar estudiantes	Sí	Por verificar
HU2 Registrar y consultar funcionarios	Sí	Por verificar
HU3 Abrir y consultar cuentas de ahorro	Sí	Por verificar
HU4 Depósitos, retiros y movimientos	Sí	Por verificar
HU5 Solicitar préstamo educativo	Sí	Por verificar
HU6 Calcular interés del préstamo (5 %)	Sí	Por verificar
HU7 Consultar préstamos y saldos pendientes	Sí	Por verificar
HU8 Pagos a capital o interés	Sí	Por verificar
HU9 Distribución proporcional de intereses	Sí	Por verificar
HU10 Estadísticas para el Rector	Sí	Por verificar

3.1 Fuera del alcance (no implementado)

Para no presentar propuestas como funcionalidades existentes, se aclara que el proyecto **no incluye**:

- Autenticación ni control de acceso por rol: “administrador”, “Rector”, “funcionario” y “sistema” son roles de las historias, pero la API no los valida.
- Endpoints `PUT`, `PATCH` o `DELETE`: las modificaciones de saldos y préstamos se hacen con `POST` sobre subrecursos (`/depositos`, `/retiros`, `/pagos`).
- Modelos o esquemas formales (Mongoose o `$jsonSchema`): las validaciones están en la capa de aplicación (`src/utils/validadores.js`).
- Pruebas automatizadas con `npm test` (no existe ese script); las pruebas se hacen con la colección de Postman.

- Interfaz gráfica, paginación y que la distribución de intereses modifique los saldos de ahorro (solo se registra).

4. Tecnologías utilizadas

Tecnología	Versión / origen	Uso en el proyecto
Node.js	>= 18 (`engines` de `package.json`)	Entorno de ejecución del backend.
Express	^4.22.3	Servidor HTTP, rutas y `express.json()` (límite 100 kb).
MongoDB (driver oficial `mongodb`)	^6.21.0	Conexión y operaciones sobre la base `cooperativa_universitaria`. No se usa Mongoose.
dotenv	^18.0.7	Lectura de variables de entorno desde `.env`.
nodemon	^3.1.14 (desarrollo)	Reinicio automático con `npm run dev`.
MongoDB Community Server / Atlas	Local o en la nube	Base de datos; el README describe servidor único, replica set local y Atlas.
MongoDB Compass	Herramienta externa	Verificar la persistencia de los datos.
Postman (o Thunder Client)	Colección `postman/cooperativa.postman_collection.json`	Demostración y pruebas de los endpoints.
Git y GitHub	Herramienta externa	Publicación del repositorio.

5. Arquitectura del sistema

La aplicación sigue una arquitectura por capas. Una petición pasa por las rutas, el controlador valida la entrada, el servicio aplica las reglas de negocio y consulta MongoDB, y la respuesta se devuelve en JSON. Cualquier error llega al manejador central.

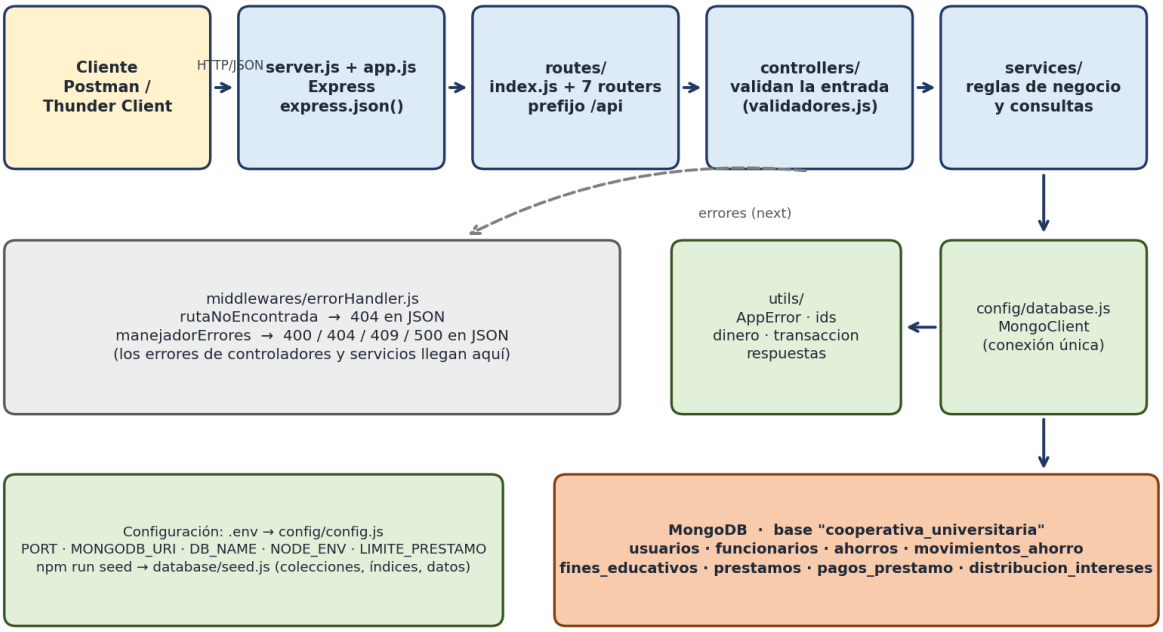


Figura A. Arquitectura por capas de `cooperativa-api` *(elaborada a partir de los archivos reales del proyecto).*

Componente	Archivos	Función
Servidor	`src/server.js`	Conecta con MongoDB antes de escuchar; si falla, informa y termina. Detecta si hay transacciones (replica set). Cierra la conexión con SIGINT/SIGTERM.
Aplicación Express	`src/app.js`	Configura `express.json`, la ruta `/`, el prefijo `/api`, el 404 y el manejador de errores.
Rutas	`src/routes/` (index + 7 routers)	Asocian método y ruta con el controlador; usan `asyncHandler` para capturar errores asíncronos.
Controladores	`src/controllers/` (6 archivos)	Validan la entrada con `validadores.js` y responden con `ok` (200) o `creado` (201).
Servicios	`src/services/` (6 archivos)	Contienen las reglas de negocio y las consultas a MongoDB.
Utilidades	`src/utils/`	`AppError` (error con código HTTP), `validadores`, `ids` (genera U001, A011…), `dinero` (redondeo a 2 decimales), `transaccion`, `respuestas`, `asyncHandler`.
Conexión	`src/config/database.js`, `config.js`	Un único `MongoClient`; la configuración sale de variables de entorno.
Errores	`src/middlewares/errorHandler.js`	Convierte cualquier error en JSON `{ success: false, message }`. En producción oculta el detalle de los errores 500.
Datos iniciales	`database/seed.js`, `datos-iniciales.js`	Crea colecciones, índices únicos y datos de prueba, sin borrar lo existente.

5.1 Manejo de errores y códigos HTTP

Código	Cuándo lo devuelve la API
200 / 201	Consulta correcta / recurso creado (`ok` y `creado` en `utils/respuestas.js`).
400	Datos inválidos, JSON mal formado, identificador con formato incorrecto o regla incumplida (saldo insuficiente, pago mayor a la deuda, monto sobre el límite).
404	Recurso inexistente o ruta no definida.
409	Duplicados o conflictos (documento/correo repetido, cuenta ya existente, préstamo activo previo, periodo ya distribuido, error 11000 de MongoDB).
500	Error interno o MongoDB no disponible. Se registra en consola y la aplicación sigue funcionando.

5.2 Consistencia de las operaciones compuestas

Depósitos, retiros, pagos y distribuciones guardan dos o más cosas a la vez (por ejemplo saldo y movimiento). `src/utils/transaccion.js` usa una **transacción real** si MongoDB corre como replica set o Atlas; en un servidor único aplica una **reversión manual** si un paso posterior falla. Esto se anuncia al iniciar el servidor (“Transacciones: ACTIVADAS” o “no disponibles”); ambos casos son válidos.

6. Historias de usuario

Las diez historias usan la estructura exigida por el PDF: “Como [rol o tipo de usuario], quiero [acción o necesidad], para [beneficio o valor obtenido]”. Provienen de `docs/historias-usuario.md`, y los endpoints se tomaron de los archivos de rutas. Cada historia coincide con una carpeta de la colección Postman.

HU1 – Registrar y consultar estudiantes

Historia de usuario	Como administrador, quiero registrar un estudiante y consultar sus datos, para permitirle utilizar los servicios de ahorro y crédito educativo.
Identificador	HU1
Rol	administrador
Necesidad	registrar un estudiante y consultar sus datos
Beneficio	permitirle utilizar los servicios de ahorro y crédito educativo
Funcionalidad implementada	Registrar y consultar estudiantes

Endpoint y método	`POST /api/estudiantes`
	`GET /api/estudiantes`
	`GET /api/estudiantes/:usuario_id`

Petición a demostrar (★)	`POST /api/estudiantes`
Resultado esperado	Estudiante guardado en `usuarios` con id nuevo (U012); duplicado → 409.
Estado de verificación	Código y colección Postman presentes (revisión estática). Ejecución con MongoDB: por verificar. Validación 400 de campos faltantes: probada sin BD.

HU2 – Registrar y consultar funcionarios

Historia de usuario	Como Rector, quiero registrar funcionarios con diferentes cargos, para delegar actividades administrativas de la cooperativa.
Identificador	HU2
Rol	Rector
Necesidad	registrar funcionarios con diferentes cargos
Beneficio	delegar actividades administrativas de la cooperativa
Funcionalidad implementada	Registrar y consultar funcionarios

Endpoint y método	`POST /api/funcionarios`
	`GET /api/funcionarios`
	`GET /api/funcionarios/:funcionario_id`

Petición a demostrar (★)	`POST /api/funcionarios`
Resultado esperado	Funcionario guardado en `funcionarios` (F006) con cargo permitido; cargo no permitido → 400.
Estado de verificación	Código y colección Postman presentes (revisión estática). Ejecución con MongoDB: por verificar. Validación 400 de cargo no permitido: probada sin BD.

HU3 – Crear y consultar cuentas de ahorro

Historia de usuario	Como estudiante, quiero abrir una cuenta de ahorro, para depositar y administrar mi dinero dentro de la cooperativa.
Identificador	HU3
Rol	estudiante
Necesidad	abrir una cuenta de ahorro
Beneficio	depositar y administrar mi dinero dentro de la cooperativa
Funcionalidad implementada	Crear y consultar cuentas de ahorro

Endpoint y método	`POST /api/ahorros`
	`GET /api/ahorros`
	`GET /api/ahorros/:ahorro_id`

Petición a demostrar (★)	`POST /api/ahorros`
Resultado esperado	Cuenta A011 con saldo 0; estudiante inexistente → 404; segunda cuenta → 409.
Estado de verificación	Código y colección Postman presentes (revisión estática). Ejecución con MongoDB: por verificar.

HU4 – Depósitos y retiros con validación de saldo

Historia de usuario	Como estudiante, quiero realizar depósitos o retiros, para mantener actualizado mi saldo de ahorro y consultar los movimientos.
Identificador	HU4
Rol	estudiante
Necesidad	realizar depósitos o retiros
Beneficio	mantener actualizado mi saldo de ahorro y consultar los movimientos
Funcionalidad implementada	Depósitos y retiros con validación de saldo

Endpoint y método	`POST /api/ahorros/:ahorro_id/depositos`
	`POST /api/ahorros/:ahorro_id/retiros`
	`GET /api/ahorros/:ahorro_id/movimientos`

Petición a demostrar (★)	`POST …/depositos y POST …/retiros`
Resultado esperado	Depósito 300.000 y retiro 100.000 → saldo 200.000 y movimientos M014 y M015; retiro mayor al saldo → 400.
Estado de verificación	Código y colección Postman presentes (revisión estática). Ejecución con MongoDB: por verificar. Validación 400 de valor negativo: probada sin BD.

HU5 – Solicitar préstamos educativos con validaciones

Historia de usuario	Como estudiante, quiero solicitar un préstamo indicando su finalidad educativa, para obtener financiación destinada exclusivamente a mis estudios.
Identificador	HU5
Rol	estudiante
Necesidad	solicitar un préstamo indicando su finalidad educativa
Beneficio	obtener financiación destinada exclusivamente a mis estudios
Funcionalidad implementada	Solicitar préstamos educativos con validaciones

Endpoint y método	`POST /api/prestamos`
	`GET /api/fines-educativos`
	`GET /api/prestamos/:prestamo_id`

Petición a demostrar (★)	`POST /api/prestamos`
Resultado esperado	Préstamo P009 con interés inicial 50.000 (5 % de 1.000.000); límite, estudiante, finalidad y préstamo activo previo se validan.
Estado de verificación	Código y colección Postman presentes (revisión estática). Ejecución con MongoDB: por verificar. Validación 400 de monto faltante: probada sin BD.

HU6 – Consultar el cálculo del interés del préstamo

Historia de usuario	Como funcionario, quiero calcular el interés del préstamo aplicando una tasa del 5 %, para conocer el valor total que debe pagar el estudiante.
Identificador	HU6
Rol	funcionario
Necesidad	calcular el interés del préstamo aplicando una tasa del 5 %
Beneficio	conocer el valor total que debe pagar el estudiante
Funcionalidad implementada	Consultar el cálculo del interés del préstamo
Endpoint y método	`GET /api/prestamos/:prestamo_id/interes`
Petición a demostrar (★)	`GET /api/prestamos/P003/interes`
Resultado esperado	Para P003: interés 150.000 y total 3.150.000 (consulta; no modifica datos).
Estado de verificación	Código y colección Postman presentes (revisión estática). Ejecución con MongoDB: por verificar.

HU7 – Consultar préstamos y saldos pendientes de un estudiante

Historia de usuario	Como estudiante, quiero consultar mis préstamos y saldos pendientes, para conocer cuánto capital e interés debo.
Identificador	HU7
Rol	estudiante
Necesidad	consultar mis préstamos y saldos pendientes
Beneficio	conocer cuánto capital e interés debo
Funcionalidad implementada	Consultar préstamos y saldos pendientes de un estudiante
Endpoint y método	`GET /api/estudiantes/:usuario_id/prestamos`
Petición a demostrar (★)	`GET /api/estudiantes/U003/prestamos`
Resultado esperado	Lista de préstamos de U003 con finalidad, capital e interés pendientes y totales.
Estado de verificación	Código y colección Postman presentes (revisión estática). Ejecución con MongoDB: por verificar. Validación 400 de filtro `estado` inválido: probada sin BD.

HU8 – Pagos diferenciados a capital o interés

Historia de usuario	Como estudiante, quiero realizar un pago y seleccionar si deseo abonarlo al capital o al interés, para controlar la forma en que disminuye mi deuda.
Identificador	HU8
Rol	estudiante
Necesidad	realizar un pago y seleccionar si deseo abonarlo al capital o al interés
Beneficio	controlar la forma en que disminuye mi deuda
Funcionalidad implementada	Pagos diferenciados a capital o interés
Endpoint y método	`POST /api/prestamos/:prestamo_id/pagos` `GET /api/prestamos/:prestamo_id/pagos`
Petición a demostrar (★)	`POST …/P003/pagos y POST …/P008/pagos`
Resultado esperado	Abono a capital 1.000.000 en P003 → capital 2.000.000 e interés 100.000; pago a interés 50.000 en P008 → interés 100.000, capital sin cambio.
Estado de verificación	Código y colección Postman presentes (revisión estática). Ejecución con MongoDB: por verificar. Validación 400 de `tipo_abono` inválido: probada sin BD.

HU9 – Calcular y registrar la distribución proporcional de intereses

Historia de usuario	Como sistema, quiero distribuir los intereses generados por los préstamos proporcionalmente entre los estudiantes ahorradores, para reconocer su participación en el fondo de la cooperativa.
Identificador	HU9
Rol	sistema
Necesidad	distribuir los intereses generados por los préstamos proporcionalmente entre los estudiantes ahorradores
Beneficio	reconocer su participación en el fondo de la cooperativa
Funcionalidad implementada	Calcular y registrar la distribución proporcional de intereses

Endpoint y método	`GET /api/distribucion-intereses/calculo`
	`POST /api/distribucion-intereses`
	`GET /api/distribucion-intereses`

Petición a demostrar (★)	`POST /api/distribucion-intereses`
Resultado esperado	Cada ahorrador recibe saldo / total ahorrado × intereses; el periodo queda guardado y no se repite (409).
Estado de verificación	Código y colección Postman presentes (revisión estática). Ejecución con MongoDB: por verificar. Validaciones 400 de periodo y de intereses faltantes: probadas sin BD.

HU10 – Estadísticas y reportes administrativos

Historia de usuario	Como Rector, quiero consultar estadísticas de ahorros, préstamos, pagos e intereses, para supervisar el funcionamiento de la cooperativa.
Identificador	HU10
Rol	Rector
Necesidad	consultar estadísticas de ahorros, préstamos, pagos e intereses
Beneficio	supervisar el funcionamiento de la cooperativa
Funcionalidad implementada	Estadísticas y reportes administrativos
Endpoint y método	`GET /api/reportes/estadisticas`
Petición a demostrar (★)	`GET /api/reportes/estadisticas`
Resultado esperado	Totales de ahorro, préstamos, pagos por tipo, intereses distribuidos y rankings.
Estado de verificación	Código y colección Postman presentes (revisión estática). Ejecución con MongoDB: por verificar.

7. Diseño de la base de datos

La base se llama `cooperativa_universitaria` (variable `DB_NAME`) y tiene **8 colecciones**. Cada documento se identifica con un identificador de negocio (por ejemplo `U001`, `A001`, `P001`), generado por la API con el siguiente número libre. Las respuestas no incluyen el `_id` interno de MongoDB.

Colección	Propósito	Campos principales	Identificador y referencias	Validaciones / índices existentes
`usuarios`	Estudiantes y el Rector.	`nombre`, `documento`, `telefono`, `email`, `tipo` (estudiante | rector), `estado`	`usuario_id` (U###)	Índices únicos: `usuario_id`, `documento`, `email`.
`funcionarios`	Personal administrativo con cargo.	`nombre`, `cargo`, `telefono`, `email`, `estado`	`funcionario_id` (F###)	Índices únicos: `funcionario_id`, `email`. Cargo validado en la API.
`ahorros`	Una cuenta de ahorro por estudiante.	`saldo`, `estado`	`ahorro_id` (A###); `usuario_id` → `usuarios`	Índices únicos: `ahorro_id`, `usuario_id`.
`movimientos_ahorro`	Historial de depósitos y retiros.	`tipo` (deposito | retiro), `valor`, `fecha`, `descripcion`	`movimiento_id` (M###); `ahorro_id` → `ahorros`; `usuario_id` → `usuarios`	Índice único: `movimiento_id`.
`fines_educativos`	Catálogo de finalidades válidas del préstamo.	`nombre`, `descripcion`, `estado`	`fin_id` (FE###)	Índice único: `fin_id`.
`prestamos`	Préstamos educativos y sus saldos.	`capital_inicial`, `interes_inicial`, `capital_pendiente`, `interes_pendiente`, `tasa_interes`, `estado` (activo | pagado), `fecha_solicitud`	`prestamo_id` (P###); `usuario_id` → `usuarios`; `fin_id` → `fines_educativos`	Índice único: `prestamo_id`. Un préstamo activo por estudiante se controla en el código.
`pagos_prestamo`	Historial de pagos a capital o interés.	`tipo_abono` (capital | interes), `valor`, `fecha`	`pago_id` (PG###); `prestamo_id` → `prestamos`; `usuario_id` → `usuarios`	Índice único: `pago_id`.
`distribucion_intereses`	Interés recibido por cada ahorrador en un periodo.	`periodo` (AAAA-MM), `ahorro_base`, `total_ahorrado`, `intereses_generados`, `participacion`, `interes_recibido`	`distribucion_id` (DI###); `usuario_id` → `usuarios`; `ahorro_id` → `ahorros`	Índice único: `distribucion_id`. Un periodo no se repite: lo controla el código.

Los índices únicos los crea `npm run seed` *(*`database/seed.js`*); la API no los crea al iniciar. Los datos iniciales son 11 usuarios (10 estudiantes y el Rector), 5 funcionarios, 10 cuentas, 13 movimientos, 9 finalidades, 8 préstamos, 3 pagos y 10 distribuciones del periodo 2026-03.*

7.1 Validaciones realmente implementadas

- **En la API** (`validadores.js`): campos obligatorios, tipo texto, longitudes, correo con formato válido, documento y teléfono solo con dígitos, montos mayores que cero, finitos y con máximo 2 decimales, listas de valores permitidos, formato de identificadores (U###, A###…) y de periodo (AAAA-MM). Se rechazan objetos en campos de texto para evitar inyección de operadores de MongoDB.
- **En MongoDB**: índices únicos creados por el seed (ver tabla). No hay `$jsonSchema`.
- **Integridad referencial**: no la impone MongoDB; los servicios comprueban que el estudiante, la cuenta, el préstamo o la finalidad existan antes de operar.

7.2 Recomendaciones futuras (no implementadas)

- Agregar validadores `$jsonSchema` al crear las colecciones en `seed.js`, para cumplir de forma más explícita con “modelos o esquemas”.
- Crear un índice único compuesto `(periodo, usuario_id)` en `distribucion_intereses` y un índice parcial para un solo préstamo activo por estudiante, de modo que la base garantice lo que hoy controla el código.
- Crear los índices al iniciar la API (y no solo con el seed) y agregar índices de consulta en `movimientos_ahorro.ahorro_id` y `pagos_prestamo.prestamo_id`.

8. Endpoints de la API REST

URL base local: `http://localhost:3000`. Todas las respuestas tienen el formato `{ "success": true|false, "message": "...", "data": {...} }`; los errores 400 agregan `errors` con la lista de campos inválidos. Los ejemplos corresponden a una base recién inicializada con `npm run seed`, ejecutando HU1 → HU10 en orden. Fuente: `src/routes/` y `src/controllers/`.

N.º	HU	Método	Endpoint	Parámetros	Cuerpo JSON de ejemplo	Resp
1	—	GET	`/`	—	—	`{ suc docum`
2	—	GET	`/api/health`	—	—	`data: "cone`
3	HU1	POST	`/api/estudiantes`	—	`{"nombre":"Daniel Esteban Molina","documento":"1001001012","telefono":"3001111012","email":"daniel.molina@universidad.edu.co"}`	Estud `tipo`
4	HU1	GET	`/api/estudiantes`	Query opcional `estado` (activo | inactivo)	—	`data:`
5	HU1	GET	`/api/estudiantes/:usuario_id`	`usuario_id` (U###)	—	Docu
6	HU2	POST	`/api/funcionarios`	—	`{"nombre":"Natalia Marcela Gómez","cargo":"Coordinador de cartera","telefono":"3102001006","email":"natalia.gomez@universidad.edu.co"}` (`estado` opcional)	Funci F006,
7	HU2	GET	`/api/funcionarios`	Query opcionales `cargo`, `estado`	—	`data:`
8	HU2	GET	`/api/funcionarios/:funcionario_id`	`funcionario_id` (F###)	—	Docu
9	HU3	POST	`/api/ahorros`	—	`{"usuario_id":"U012"}`	Cuen `estad`
10	HU3	GET	`/api/ahorros`	—	—	`data:`
11	HU3	GET	`/api/ahorros/:ahorro_id`	`ahorro_id` (A###)	—	Docu
12	HU4	POST	`/api/ahorros/:ahorro_id/depositos`	`ahorro_id`	`{"valor":300000,"descripcion":"Depósito realizado por el estudiante"}` (`descripcion` opcional)	`data: saldo 30000`
13	HU4	POST	`/api/ahorros/:ahorro_id/retiros`	`ahorro_id`	`{"valor":100000}`	`data: saldo saldo`
14	HU4	GET	`/api/ahorros/:ahorro_id/movimientos`	`ahorro_id`; query opcional `tipo` (deposito | retiro)	—	`data: movim`

N.º	HU	Método	Endpoint	Parámetros	Cuerpo JSON de ejemplo	Resp
15	HU5	GET	`/api/fines-educativos`	—	—	`data:` `}` (apo
16	HU5	POST	`/api/prestamos`	—	`{"usuario_id":"U009","fin_id":"FE009","monto":1000000}`	Prést `inter` 0.05,
17	HU5	GET	`/api/prestamos/:prestamo_id`	`prestamo_id` (P###)	—	Docu
18	HU6	GET	`/api/prestamos/:prestamo_id/interes`	`prestamo_id`	—	P003 intere 3150 `inter`
19	HU7	GET	`/api/estudiantes/:usuario_id/prestamos`	`usuario_id`; query opcional `estado` (activo | pagado)	—	`data:` `canti` `total` `total` `}`
20	HU8	POST	`/api/prestamos/:prestamo_id/pagos`	`prestamo_id`	`{"tipo_abono":"capital","valor":1000000}` (`tipo_abono`: capital | interes)	`data:` `saldo`
21	HU8	GET	`/api/prestamos/:prestamo_id/pagos`	`prestamo_id`	—	`data:` `pagos`
22	HU9	GET	`/api/distribucion-intereses/calculo`	Query obligatoria `intereses_generados`	—	Cálcu `suma_` `difer` `distr`
23	HU9	POST	`/api/distribucion-intereses`	—	`{"periodo":"2026-04","intereses_generados":500000}`	Distrib ahorra
24	HU9	GET	`/api/distribucion-intereses`	Query opcionales `periodo` (AAAA-MM), `usuario_id`	—	`data:` `total` `distr`
25	HU10	GET	`/api/reportes/estadisticas`	—	—	`estud` `inter` `estud` `estud`

Los endpoints 1, 2, 15 y 17 son de apoyo (no son una de las 10 funcionalidades). Los códigos de la última columna son todos los que el código puede devolver; el esperado en el caso exitoso es el primero. Ninguno de los endpoints está sin implementar; su funcionamiento con MongoDB está ***por verificar****. No existen* `PUT`, `PATCH` *ni* `DELETE`.

9. Reglas de negocio

Todas las reglas siguientes **están implementadas en el código** (servicios y validadores). Lo que es solo propuesta se lista al final.

9.1 Registro de estudiantes y funcionarios

- Estudiante: nombre (3–100 caracteres), documento (5–15 dígitos), teléfono (7–15 dígitos) y correo válido. El `usuario_id` (U###) se genera automáticamente; `tipo` = estudiante y `estado` = activo.
- No se permite repetir documento ni correo (409).
- Funcionario: el cargo debe ser uno de Director financiero, Asesor de créditos, Tesorero, Auxiliar administrativo, Analista financiero o Coordinador de cartera. El `estado` es activo por defecto y el correo no puede repetirse.

9.2 Apertura y administración de ahorros

- La cuenta se abre solo para un estudiante existente y activo, con saldo inicial 0. Cada estudiante tiene una sola cuenta (409 si ya existe).

9.3 Depósitos y retiros

- El valor debe ser un número mayor que cero, con máximo 2 decimales (no se aceptan texto, cero ni negativos).
- El depósito suma al saldo; el retiro solo se aplica si `saldo >= valor` (400 “Saldo insuficiente”). La cuenta debe estar activa.
- Cada operación guarda el cambio de saldo y un movimiento (M###) en `movimientos_ahorro` como una operación atómica (transacción o reversión manual).

9.4 Solicitud de préstamos educativos

- Exige estudiante activo, finalidad educativa activa, monto > 0 y menor o igual al límite (5.000.000 por defecto, variable `LIMITE_PRESTAMO`).
- Un estudiante solo puede tener un préstamo activo (409). El préstamo nace con `capital_pendiente` = monto e `interes_pendiente` = `interes_inicial`.

9.5 Cálculo de intereses

- Tasa fija del 5 % (`tasaInteres` en `config.js`; no se cambia por variable de entorno). Interés inicial = capital × 0,05, redondeado a 2 decimales.
- `GET /api/prestamos/:prestamo_id/interes` devuelve dos cálculos: sobre el capital inicial y sobre el capital pendiente, junto con el interés pendiente registrado. Es una consulta: no modifica datos.

9.6 Pagos a capital e intereses

- **Pago a capital**: `capital_pendiente` baja en el valor pagado (no puede superarlo) y el `interes_pendiente` se recalcula como nuevo capital × 5 %.
- **Pago a interés**: baja el `interes_pendiente` (no puede superarlo); el capital no cambia.

- Si capital e interés quedan en 0, el préstamo pasa a `pagado`; un préstamo pagado no admite más pagos (400). Cada pago se guarda en `pagos_prestamo` (PG###) junto con la actualización del préstamo.
- Si otra solicitud modificó el préstamo entre la lectura y la escritura, el pago se rechaza con 409.

9.7 Distribución proporcional de intereses

- Participan las cuentas activas con saldo mayor que cero. `participacion` = saldo ÷ total ahorrado e `interes_recibido` = participación × intereses generados, redondeado a 2 decimales.
- Se guarda un documento por ahorrador en `distribucion_intereses`; un periodo (AAAA-MM) solo puede distribuirse una vez (409). **No modifica saldos**.
- El redondeo puede dejar una diferencia de centavos; se informa en `diferencia_por_redondeo` y no se ajusta en silencio.

9.8 Reportes administrativos

- `GET /api/reportes/estadisticas` calcula con `aggregate` (`$match`, `$group`): cantidad de estudiantes, total ahorrado (cuentas activas), préstamos (cantidad, total prestado, intereses generados, capital e intereses pendientes de préstamos activos, cantidad por estado), pagos por tipo con promedio, intereses distribuidos por periodo y los cinco estudiantes con mayor ahorro y los préstamos activos.

9.9 Propuestas no implementadas

- Autenticación y autorización por rol, ajuste automático del redondeo, abonar los intereses distribuidos al saldo de ahorro, edición o anulación de movimientos y pagos, y tasa configurable.

11. Pruebas funcionales con Postman

11.1 Preparación

- Servidor en ejecución y base recién inicializada: `npm run seed` y `npm start`.
- En Postman: **Import** → `postman/cooperativa.postman_collection.json`. La variable `baseUrl` vale `http://localhost:3000`; los ids de los recursos creados se guardan automáticamente en variables de colección (`usuario_nuevo`, `ahorro_nuevo`, `prestamo_nuevo`…).
- Ejecutar las carpetas en orden HU1 → HU10; las peticiones marcadas con ★ son las que se muestran en la sustentación.
- **Reproducibilidad**: la colección crea datos (U012, F006, A011, P009, PG004, PG005 y la distribución 2026-04). Para repetirla hay que reiniciar la base (en Compass, **Drop Database** sobre `cooperativa_universitaria`) y volver a ejecutar `npm run seed`; de lo contrario, las peticiones de creación responden 409.

11.2 Pruebas funcionales (10 funcionalidades)

Cada ficha indica lo que debe ejecutarse. Los recuadros de captura de Postman se insertan debajo de la ficha; los de MongoDB Compass están en la sección 12. Todas las pruebas están **pendientes de ejecución real** con MongoDB.

P01 – HU1: Registrar estudiante

Número y nombre	P01 – Registrar estudiante
Historia de usuario	HU1: Como administrador, quiero registrar un estudiante y consultar sus datos, para permitirle utilizar los servicios de ahorro y crédito educativo.
Método HTTP y URL	`POST http://localhost:3000/api/estudiantes`
Datos de entrada	`{ "nombre": "Daniel Esteban Molina", "documento": "1001001012", "telefono": "3001111012", "email": "daniel.molina@universidad.edu.co" }`
Pasos	1. Abrir en Postman la carpeta de HU1 y seleccionar la petición ★ (1 petición). 2. Pulsar **Send** y revisar cuerpo, respuesta y código. 3. Verificar la persistencia en MongoDB Compass si aplica.
Resultado esperado	Estudiante creado con `usuario_id` = "U012", tipo "estudiante", estado "activo".
Código HTTP esperado	201
Persistencia en MongoDB	Colección `usuarios`, filtro `{ "usuario_id": "U012" }`.
Capturas	E04 (Postman) · E05 (Compass, sección 12)

[ESPACIO RESERVADO PARA LA CAPTURA 04] · E04	
Nombre de la evidencia	HU1 – Registro de estudiante (POST)
Qué debo capturar	`POST /api/estudiantes` con el cuerpo JSON y la respuesta 201.
Dónde debo hacerlo	Postman, con el servidor y MongoDB en ejecución.
Pasos para obtenerla	1. Iniciar el servidor (`npm start`) con la base recién inicializada (`npm run seed`) y respetando el orden HU1 → HU10. 2. En Postman, abrir la carpeta “HU1 - Registrar y consultar estudiantes” de la colección importada y seleccionar la petición ★ “Registrar estudiante (válido)”. 3. Pulsar **Send**. 4. Confirmar que el código y el JSON coinciden con lo esperado (ver ficha de la prueba). 5. Capturar la ventana completa de la petición y la respuesta, con zoom suficiente para leer.
Qué debe verse en la imagen	Método POST, URL, cuerpo JSON (Daniel Esteban Molina), respuesta con `usuario_id: "U012"`, `tipo: "estudiante"`, `estado: "activo"` y código 201 Created.
Nombre sugerido del archivo	`evidencia_04_hu1_registro_estudiante.png`

[INSERTAR AQUÍ LA IMAGEN REAL]

Figura 04. HU1 – Registro de estudiante (POST)

P02 – HU2: Registrar funcionario

Número y nombre	P02 – Registrar funcionario
Historia de usuario	HU2: Como Rector, quiero registrar funcionarios con diferentes cargos, para delegar actividades administrativas de la cooperativa.
Método HTTP y URL	`POST http://localhost:3000/api/funcionarios`
Datos de entrada	`{ "nombre": "Natalia Marcela Gómez", "cargo": "Coordinador de cartera", "telefono": "3102001006", "email": "natalia.gomez@universidad.edu.co" }`
Pasos	1. Abrir en Postman la carpeta de HU2 y seleccionar la petición ★ (1 petición). 2. Pulsar **Send** y revisar cuerpo, respuesta y código. 3. Verificar la persistencia en MongoDB Compass si aplica.
Resultado esperado	`funcionario_id` = "F006", estado "activo".
Código HTTP esperado	201
Persistencia en MongoDB	Colección `funcionarios`, filtro `{ "funcionario_id": "F006" }`.
Capturas	E06 · E07

[ESPACIO RESERVADO PARA LA CAPTURA 06] · E06	
Nombre de la evidencia	HU2 – Registro de funcionario (POST)
Qué debo capturar	`POST /api/funcionarios` con cargo permitido y respuesta 201.
Dónde debo hacerlo	Postman, con el servidor y MongoDB en ejecución.
Pasos para obtenerla	1. Iniciar el servidor (`npm start`) con la base recién inicializada (`npm run seed`) y respetando el orden HU1 → HU10. 2. En Postman, abrir la carpeta “HU2 - Registrar y consultar funcionarios” de la colección importada y seleccionar la petición ★ “Registrar funcionario (válido)”. 3. Pulsar **Send**. 4. Confirmar que el código y el JSON coinciden con lo esperado (ver ficha de la prueba). 5. Capturar la ventana completa de la petición y la respuesta, con zoom suficiente para leer.
Qué debe verse en la imagen	Método POST, URL, cuerpo JSON (cargo “Coordinador de cartera”), `funcionario_id: "F006"` y código 201 Created.
Nombre sugerido del archivo	`evidencia_06_hu2_registro_funcionario.png`

[INSERTAR AQUÍ LA IMAGEN REAL]

Figura 06. HU2 – Registro de funcionario (POST)

P03 – HU3: Abrir cuenta de ahorro

Número y nombre	P03 – Abrir cuenta de ahorro
Historia de usuario	HU3: Como estudiante, quiero abrir una cuenta de ahorro, para depositar y administrar mi dinero dentro de la cooperativa.
Método HTTP y URL	`POST http://localhost:3000/api/ahorros`
Datos de entrada	`{ "usuario_id": "U012" }`
Pasos	1. Abrir en Postman la carpeta de HU3 y seleccionar la petición ★ (1 petición). 2. Pulsar **Send** y revisar cuerpo, respuesta y código. 3. Verificar la persistencia en MongoDB Compass si aplica.
Resultado esperado	`ahorro_id` = "A011", `saldo` = 0.
Código HTTP esperado	201
Persistencia en MongoDB	Colección `ahorros`, filtro `{ "ahorro_id": "A011" }`.
Capturas	E08 · E09

[ESPACIO RESERVADO PARA LA CAPTURA 08] · E08	
Nombre de la evidencia	HU3 – Apertura de cuenta de ahorro (POST)
Qué debo capturar	`POST /api/ahorros` con `{ "usuario_id": "U012" }` y respuesta 201.
Dónde debo hacerlo	Postman, con el servidor y MongoDB en ejecución.
Pasos para obtenerla	1. Iniciar el servidor (`npm start`) con la base recién inicializada (`npm run seed`) y respetando el orden HU1 → HU10. 2. En Postman, abrir la carpeta “HU3 - Crear y consultar cuentas de ahorro” de la colección importada y seleccionar la petición ★ “Abrir cuenta de ahorro (válido)”. 3. Pulsar **Send**. 4. Confirmar que el código y el JSON coinciden con lo esperado (ver ficha de la prueba). 5. Capturar la ventana completa de la petición y la respuesta, con zoom suficiente para leer.
Qué debe verse en la imagen	Método POST, URL, cuerpo JSON, `ahorro_id: "A011"`, `saldo: 0` y código 201 Created.
Nombre sugerido del archivo	`evidencia_08_hu3_apertura_cuenta.png`

[INSERTAR AQUÍ LA IMAGEN REAL]

Figura 08. HU3 – Apertura de cuenta de ahorro (POST)

P04 – HU4: Depósito y retiro

Número y nombre	P04 – Depósito y retiro
Historia de usuario	HU4: Como estudiante, quiero realizar depósitos o retiros, para mantener actualizado mi saldo de ahorro y consultar los movimientos.
Método HTTP y URL	`POST http://localhost:3000/api/ahorros/A011/depositos` `POST http://localhost:3000/api/ahorros/A011/retiros`
Datos de entrada	Depósito: `{ "valor": 300000 }`. Retiro: `{ "valor": 100000 }`.
Pasos	1. Abrir en Postman la carpeta de HU4 y seleccionar la petición ★ (2 peticiones). 2. Pulsar **Send** y revisar cuerpo, respuesta y código. 3. Verificar la persistencia en MongoDB Compass si aplica.
Resultado esperado	Depósito: `saldo_actual` 300000 (movimiento M014). Retiro: `saldo_actual` 200000 (movimiento M015).
Código HTTP esperado	201 en ambos
Persistencia en MongoDB	`movimientos_ahorro` con `{ "ahorro_id": "A011" }` y saldo 200000 en `ahorros`.
Capturas	E10 · E11 · E12

[ESPACIO RESERVADO PARA LA CAPTURA 10] · E10	
Nombre de la evidencia	HU4 – Depósito de 300.000 (POST)
Qué debo capturar	`POST /api/ahorros/A011/depositos` y su respuesta 201.
Dónde debo hacerlo	Postman, con el servidor y MongoDB en ejecución.
Pasos para obtenerla	1. Iniciar el servidor (`npm start`) con la base recién inicializada (`npm run seed`) y respetando el orden HU1 → HU10. 2. En Postman, abrir la carpeta “HU4 - Depósitos y retiros” de la colección importada y seleccionar la petición ★ “Registrar depósito de 300.000”. 3. Pulsar **Send**. 4. Confirmar que el código y el JSON coinciden con lo esperado (ver ficha de la prueba). 5. Capturar la ventana completa de la petición y la respuesta, con zoom suficiente para leer.
Qué debe verse en la imagen	Método POST, URL con `A011`, cuerpo `{ "valor": 300000 }`, `movimiento_id: "M014"`, `saldo_anterior: 0`, `saldo_actual: 300000` y código 201.
Nombre sugerido del archivo	`evidencia_10_hu4_deposito.png`

[INSERTAR AQUÍ LA IMAGEN REAL]

Figura 10. HU4 – Depósito de 300.000 (POST)

[ESPACIO RESERVADO PARA LA CAPTURA 11] · E11	
Nombre de la evidencia	HU4 – Retiro de 100.000 (POST)
Qué debo capturar	`POST /api/ahorros/A011/retiros` y su respuesta 201.
Dónde debo hacerlo	Postman, con el servidor y MongoDB en ejecución.
Pasos para obtenerla	1. Iniciar el servidor (`npm start`) con la base recién inicializada (`npm run seed`) y respetando el orden HU1 → HU10. 2. En Postman, abrir la carpeta “HU4 - Depósitos y retiros” de la colección importada y seleccionar la petición ★ “Registrar retiro de 100.000”. 3. Pulsar **Send**. 4. Confirmar que el código y el JSON coinciden con lo esperado (ver ficha de la prueba). 5. Capturar la ventana completa de la petición y la respuesta, con zoom suficiente para leer.
Qué debe verse en la imagen	Método POST, URL, cuerpo `{ "valor": 100000 }`, `movimiento_id: "M015"`, `saldo_anterior: 300000`, `saldo_actual: 200000` y código 201.
Nombre sugerido del archivo	`evidencia_11_hu4_retiro.png`

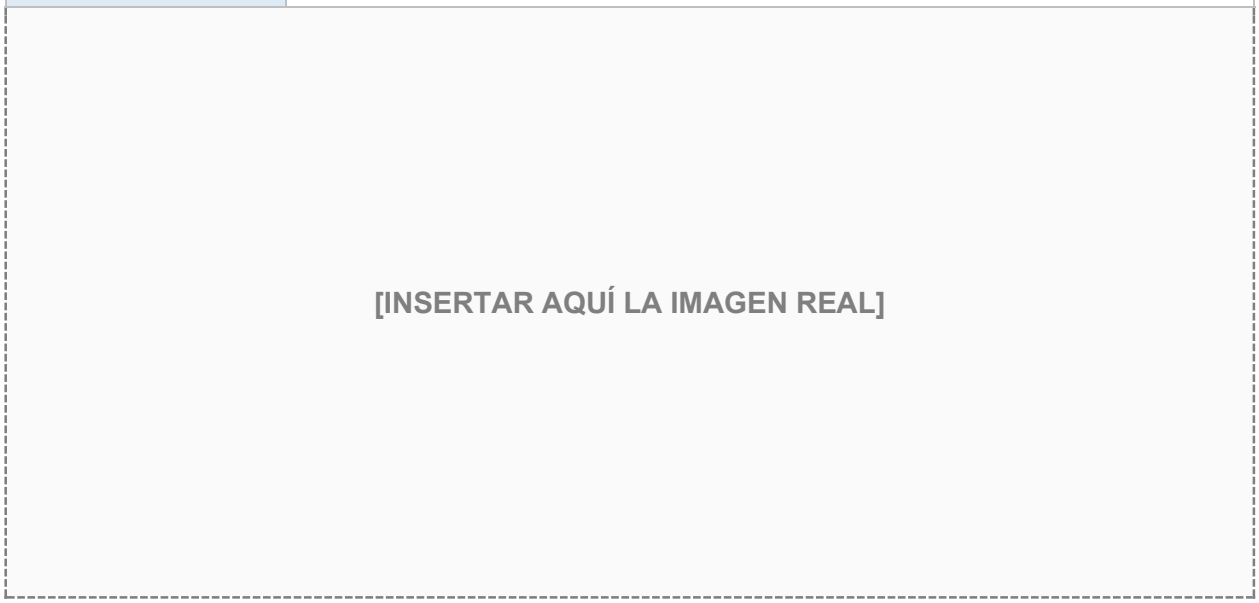


Figura 11. HU4 – Retiro de 100.000 (POST)

P05 – HU5: Solicitar préstamo educativo

Número y nombre	P05 – Solicitar préstamo educativo
Historia de usuario	HU5: Como estudiante, quiero solicitar un préstamo indicando su finalidad educativa, para obtener financiación destinada exclusivamente a mis estudios.
Método HTTP y URL	`POST http://localhost:3000/api/prestamos`
Datos de entrada	`{ "usuario_id": "U009", "fin_id": "FE009", "monto": 1000000 }`
Pasos	1. Abrir en Postman la carpeta de HU5 y seleccionar la petición ★ (1 petición). 2. Pulsar **Send** y revisar cuerpo, respuesta y código. 3. Verificar la persistencia en MongoDB Compass si aplica.
Resultado esperado	`prestamo_id` = "P009", `interes_inicial` = 50000, `tasa_interes` = 0.05. Repetir la petición responde 409.
Código HTTP esperado	201
Persistencia en MongoDB	Colección `prestamos`, filtro `{ "prestamo_id": "P009" }`.
Capturas	E13 · E14

[ESPACIO RESERVADO PARA LA CAPTURA 13] · E13	
Nombre de la evidencia	HU5 – Solicitud de préstamo educativo (POST)
Qué debo capturar	`POST /api/prestamos` con U009, FE009 y 1.000.000, respuesta 201.
Dónde debo hacerlo	Postman, con el servidor y MongoDB en ejecución.
Pasos para obtenerla	1. Iniciar el servidor (`npm start`) con la base recién inicializada (`npm run seed`) y respetando el orden HU1 → HU10. 2. En Postman, abrir la carpeta “HU5 - Solicitar préstamos educativos” de la colección importada y seleccionar la petición ★ “Solicitar préstamo (válido)”. 3. Pulsar **Send**. 4. Confirmar que el código y el JSON coinciden con lo esperado (ver ficha de la prueba). 5. Capturar la ventana completa de la petición y la respuesta, con zoom suficiente para leer.
Qué debe verse en la imagen	Método POST, URL, cuerpo JSON, `prestamo_id: "P009"`, `interes_inicial: 50000`, `tasa_interes: 0.05` y código 201 Created.
Nombre sugerido del archivo	`evidencia_13_hu5_solicitud_prestamo.png`

[INSERTAR AQUÍ LA IMAGEN REAL]

Figura 13. HU5 – Solicitud de préstamo educativo (POST)

P06 – HU6: Calcular interés del préstamo

Número y nombre	P06 – Calcular interés del préstamo
Historia de usuario	HU6: Como funcionario, quiero calcular el interés del préstamo aplicando una tasa del 5 %, para conocer el valor total que debe pagar el estudiante.
Método HTTP y URL	`GET http://localhost:3000/api/prestamos/P003/interes`
Datos de entrada	Sin cuerpo.
Pasos	1. Abrir en Postman la carpeta de HU6 y seleccionar la petición ★ (1 petición). 2. Pulsar **Send** y revisar cuerpo, respuesta y código. 3. Verificar la persistencia en MongoDB Compass si aplica.
Resultado esperado	`sobre_capital_inicial`: interés 150000 y total 3150000.
Código HTTP esperado	200
Persistencia en MongoDB	No aplica (consulta). Opcional: P003 no cambia.
Capturas	E15

[ESPACIO RESERVADO PARA LA CAPTURA 15] · E15	
Nombre de la evidencia	HU6 – Cálculo del interés del préstamo P003 (GET)
Qué debo capturar	`GET /api/prestamos/P003/interes`.
Dónde debo hacerlo	Postman, con el servidor y MongoDB en ejecución.
Pasos para obtenerla	1. Iniciar el servidor (`npm start`) con la base recién inicializada (`npm run seed`) y respetando el orden HU1 → HU10. 2. En Postman, abrir la carpeta “HU6 - Cálculo del interés del préstamo” de la colección importada y seleccionar la petición ★ “Calcular interés del préstamo P003”. 3. Pulsar **Send**. 4. Confirmar que el código y el JSON coinciden con lo esperado (ver ficha de la prueba). 5. Capturar la ventana completa de la petición y la respuesta, con zoom suficiente para leer.
Qué debe verse en la imagen	Método GET, URL, `sobre_capital_inicial` con interés 150000 y total 3150000, y código 200 OK.
Nombre sugerido del archivo	`evidencia_15_hu6_calculo_interes.png`



Figura 15. HU6 – Cálculo del interés del préstamo P003 (GET)

P07 – HU7: Consultar préstamos y saldos pendientes

Número y nombre	P07 – Consultar préstamos y saldos pendientes
Historia de usuario	HU7: Como estudiante, quiero consultar mis préstamos y saldos pendientes, para conocer cuánto capital e interés debo.
Método HTTP y URL	`GET http://localhost:3000/api/estudiantes/U003/prestamos`
Datos de entrada	Sin cuerpo.
Pasos	1. Abrir en Postman la carpeta de HU7 y seleccionar la petición ★ (1 petición). 2. Pulsar **Send** y revisar cuerpo, respuesta y código. 3. Verificar la persistencia en MongoDB Compass si aplica.
Resultado esperado	P003 con finalidad “Compra de libros”, capital pendiente 3000000 e interés pendiente 150000 (si HU8 ya se ejecutó: 2000000 y 100000).
Código HTTP esperado	200
Persistencia en MongoDB	No aplica (consulta). Comparar con `prestamos` filtrado por `{ "usuario_id": "U003" }`.
Capturas	E16

[ESPACIO RESERVADO PARA LA CAPTURA 16] · E16	
Nombre de la evidencia	HU7 – Préstamos y saldos pendientes de U003 (GET)
Qué debo capturar	`GET /api/estudiantes/U003/prestamos`.
Dónde debo hacerlo	Postman, con el servidor y MongoDB en ejecución.
Pasos para obtenerla	1. Iniciar el servidor (`npm start`) con la base recién inicializada (`npm run seed`) y respetando el orden HU1 → HU10. 2. En Postman, abrir la carpeta “HU7 - Consultar préstamos y saldos pendientes” de la colección importada y seleccionar la petición ★ “Consultar préstamos de U003”. 3. Pulsar **Send**. 4. Confirmar que el código y el JSON coinciden con lo esperado (ver ficha de la prueba). 5. Capturar la ventana completa de la petición y la respuesta, con zoom suficiente para leer.
Qué debe verse en la imagen	Método GET, URL, préstamo P003 con finalidad “Compra de libros”, capital pendiente 3000000, interés pendiente 150000 y código 200 OK (si HU8 ya se ejecutó: 2000000 y 100000).
Nombre sugerido del archivo	`evidencia_16_hu7_prestamos_estudiante.png`

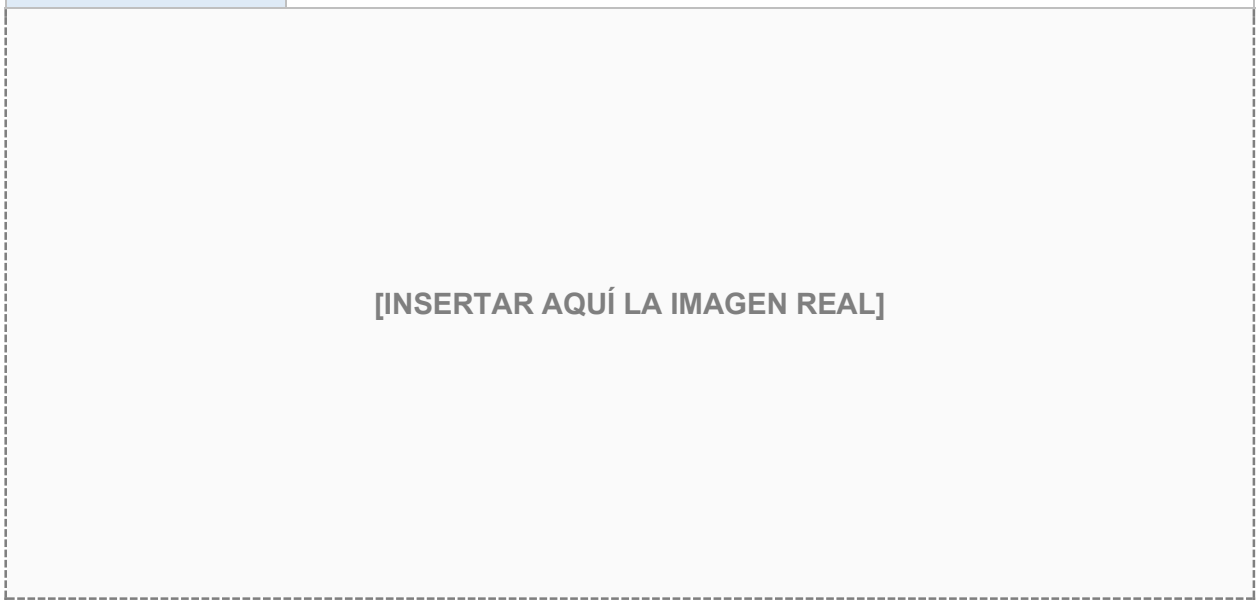


Figura 16. HU7 – Préstamos y saldos pendientes de U003 (GET)

P08 – HU8: Pago a capital y pago a interés

Número y nombre	P08 – Pago a capital y pago a interés
Historia de usuario	HU8: Como estudiante, quiero realizar un pago y seleccionar si deseo abonarlo al capital o al interés, para controlar la forma en que disminuye mi deuda.
Método HTTP y URL	`POST http://localhost:3000/api/prestamos/P003/pagos` `POST http://localhost:3000/api/prestamos/P008/pagos`
Datos de entrada	P003: `{ "tipo_abono": "capital", "valor": 1000000 }`. P008: `{ "tipo_abono": "interes", "valor": 50000 }`.
Pasos	1. Abrir en Postman la carpeta de HU8 y seleccionar la petición ★ (2 peticiones). 2. Pulsar **Send** y revisar cuerpo, respuesta y código. 3. Verificar la persistencia en MongoDB Compass si aplica.
Resultado esperado	P003: capital 2000000 e interés 100000 (pago PG004). P008: capital 3000000 e interés 100000 (pago PG005).
Código HTTP esperado	201 en ambos
Persistencia en MongoDB	`prestamos` (P003, P008) y `pagos_prestamo` (PG004, PG005).
Capturas	E17 · E18 · E19

[ESPACIO RESERVADO PARA LA CAPTURA 17] · E17	
Nombre de la evidencia	HU8 – Abono a capital de 1.000.000 en P003 (POST)
Qué debo capturar	`POST /api/prestamos/P003/pagos` con `tipo_abono: "capital"`.
Dónde debo hacerlo	Postman, con el servidor y MongoDB en ejecución.
Pasos para obtenerla	1. Iniciar el servidor (`npm start`) con la base recién inicializada (`npm run seed`) y respetando el orden HU1 → HU10. 2. En Postman, abrir la carpeta “HU8 - Pagos a capital o interés” de la colección importada y seleccionar la petición ★ “Abono a capital de 1.000.000 (P003)”. 3. Pulsar **Send**. 4. Confirmar que el código y el JSON coinciden con lo esperado (ver ficha de la prueba). 5. Capturar la ventana completa de la petición y la respuesta, con zoom suficiente para leer.
Qué debe verse en la imagen	Método POST, URL, cuerpo JSON, `pago_id: "PG004"`, préstamo con capital_pendiente 2000000 e interes_pendiente 100000 y código 201.
Nombre sugerido del archivo	`evidencia_17_hu8_pago_capital.png`

[INSERTAR AQUÍ LA IMAGEN REAL]

Figura 17. HU8 – Abono a capital de 1.000.000 en P003 (POST)

[ESPACIO RESERVADO PARA LA CAPTURA 18] · E18	
Nombre de la evidencia	HU8 – Pago a interés de 50.000 en P008 (POST)
Qué debo capturar	`POST /api/prestamos/P008/pagos` con `tipo_abono: "interes"`.
Dónde debo hacerlo	Postman, con el servidor y MongoDB en ejecución.
Pasos para obtenerla	1. Iniciar el servidor (`npm start`) con la base recién inicializada (`npm run seed`) y respetando el orden HU1 → HU10. 2. En Postman, abrir la carpeta “HU8 - Pagos a capital o interés” de la colección importada y seleccionar la petición ★ “Pago a interés de 50.000 (P008)”. 3. Pulsar **Send**. 4. Confirmar que el código y el JSON coinciden con lo esperado (ver ficha de la prueba). 5. Capturar la ventana completa de la petición y la respuesta, con zoom suficiente para leer.
Qué debe verse en la imagen	Método POST, URL, cuerpo JSON, `pago_id: "PG005"`, capital_pendiente 3000000 (sin cambio), interes_pendiente 100000 y código 201.
Nombre sugerido del archivo	`evidencia_18_hu8_pago_interes.png`

[INSERTAR AQUÍ LA IMAGEN REAL]

Figura 18. HU8 – Pago a interés de 50.000 en P008 (POST)

P09 – HU9: Distribuir intereses del periodo 2026-04

Número y nombre	P09 – Distribuir intereses del periodo 2026-04
Historia de usuario	HU9: Como sistema, quiero distribuir los intereses generados por los préstamos proporcionalmente entre los estudiantes ahorradores, para reconocer su participación en el fondo de la cooperativa.
Método HTTP y URL	`POST http://localhost:3000/api/distribucion-intereses`
Datos de entrada	`{ "periodo": "2026-04", "intereses_generados": 500000 }`
Pasos	1. Abrir en Postman la carpeta de HU9 y seleccionar la petición ★ (1 petición). 2. Pulsar **Send** y revisar cuerpo, respuesta y código. 3. Verificar la persistencia en MongoDB Compass si aplica.
Resultado esperado	Un registro por ahorrador (11 con los datos de la secuencia); cada `interes_recibido` = saldo ÷ total ahorrado × 500000. Repetir el periodo responde 409.
Código HTTP esperado	201
Persistencia en MongoDB	Colección `distribucion_intereses`, filtro `{ "periodo": "2026-04" }`.
Capturas	E20 · E21

[ESPACIO RESERVADO PARA LA CAPTURA 20] · E20	
Nombre de la evidencia	HU9 – Registro de la distribución de intereses 2026-04 (POST)
Qué debo capturar	`POST /api/distribucion-intereses` con periodo 2026-04 y 500.000.
Dónde debo hacerlo	Postman, con el servidor y MongoDB en ejecución.
Pasos para obtenerla	1. Iniciar el servidor (`npm start`) con la base recién inicializada (`npm run seed`) y respetando el orden HU1 → HU10. 2. En Postman, abrir la carpeta “HU9 - Distribución proporcional de intereses” de la colección importada y seleccionar la petición ★ “Registrar distribución del periodo 2026-04”. 3. Pulsar **Send**. 4. Confirmar que el código y el JSON coinciden con lo esperado (ver ficha de la prueba). 5. Capturar la ventana completa de la petición y la respuesta, con zoom suficiente para leer.
Qué debe verse en la imagen	Método POST, URL, cuerpo JSON, `cantidad_ahorradores`, `suma_distribuida`, `diferencia_por_redondeo`, lista `distribucion` y código 201 Created.
Nombre sugerido del archivo	`evidencia_20_hu9_distribucion_intereses.png`

[INSERTAR AQUÍ LA IMAGEN REAL]

Figura 20. HU9 – Registro de la distribución de intereses 2026-04 (POST)

P10 – HU10: Estadísticas para el Rector

Número y nombre	P10 – Estadísticas para el Rector
Historia de usuario	HU10: Como Rector, quiero consultar estadísticas de ahorros, préstamos, pagos e intereses, para supervisar el funcionamiento de la cooperativa.
Método HTTP y URL	`GET http://localhost:3000/api/reportes/estadisticas`
Datos de entrada	Sin cuerpo.
Pasos	1. Abrir en Postman la carpeta de HU10 y seleccionar la petición ★ (1 petición). 2. Pulsar **Send** y revisar cuerpo, respuesta y código. 3. Verificar la persistencia en MongoDB Compass si aplica.
Resultado esperado	Totales de estudiantes, ahorro, préstamos, pagos por tipo, intereses distribuidos y rankings.
Código HTTP esperado	200
Persistencia en MongoDB	No aplica (consulta). Comparar totales con `ahorros`, `prestamos` y `pagos_prestamo` (valores de referencia en el Anexo C).
Capturas	E22

[ESPACIO RESERVADO PARA LA CAPTURA 22] · E22	
Nombre de la evidencia	HU10 – Estadísticas para el Rector (GET)
Qué debo capturar	`GET /api/reportes/estadisticas`.
Dónde debo hacerlo	Postman, con el servidor y MongoDB en ejecución.
Pasos para obtenerla	1. Iniciar el servidor (`npm start`) con la base recién inicializada (`npm run seed`) y respetando el orden HU1 → HU10. 2. En Postman, abrir la carpeta “HU10 - Estadísticas para el Rector” de la colección importada y seleccionar la petición ★ “Consultar estadísticas de la cooperativa”. 3. Pulsar **Send**. 4. Confirmar que el código y el JSON coinciden con lo esperado (ver ficha de la prueba). 5. Capturar la ventana completa de la petición y la respuesta, con zoom suficiente para leer.
Qué debe verse en la imagen	Método GET, URL, secciones estudiantes, ahorros, préstamos, pagos, intereses distribuidos y rankings, y código 200 OK. (Si no cabe, capturar el inicio y el final de la respuesta.)
Nombre sugerido del archivo	`evidencia_22_hu10_estadisticas.png`

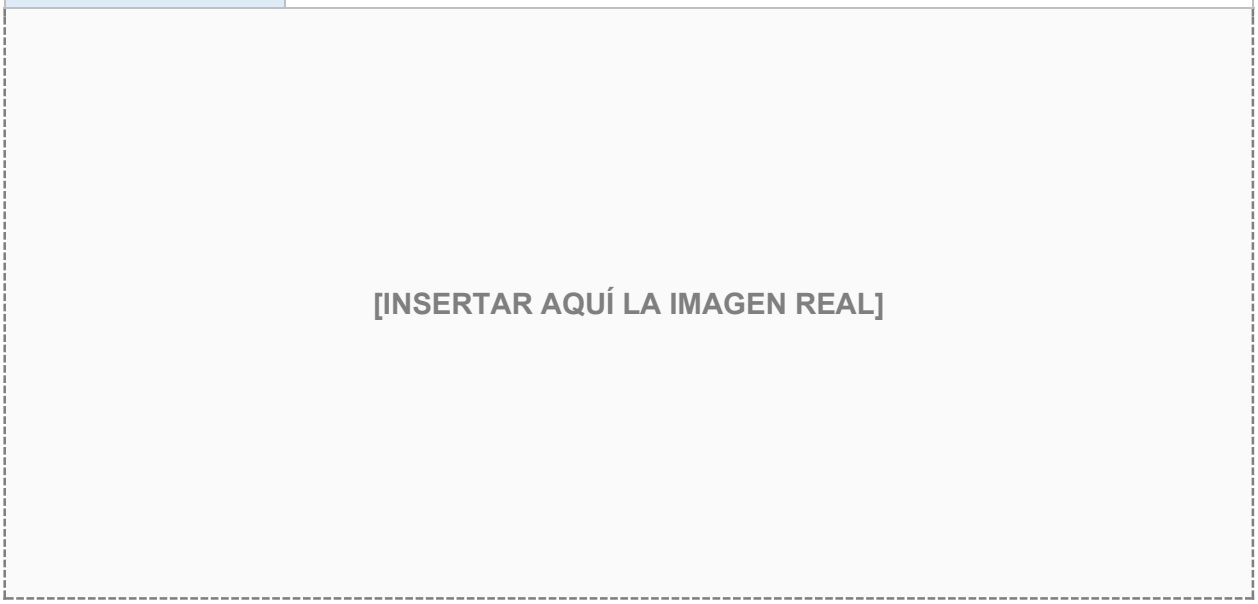


Figura 22. HU10 – Estadísticas para el Rector (GET)

. Error 400 – validación de datos (campos obligatorios)

16. Conclusiones

- El proyecto implementa en código las diez funcionalidades del MVP, organizadas en rutas, controladores y servicios, con respuestas JSON uniformes, un manejador central de errores y documentación (README, historias y colección de Postman) que este informe unifica y completa.
- Las validaciones y los errores 400 y 404 se comportaron como se espera en las pruebas preliminares sin base de datos, y la aplicación no se detuvo ante peticiones inválidas ni ante la falta de MongoDB.
- La persistencia en MongoDB, el seed y las respuestas exitosas **aún no se han demostrado**; este documento deja listas las pruebas, los filtros de Compass y los espacios de captura para hacerlo.
- Los principales riesgos frente al PDF son de demostración y no de código: ejecutar las 10 funcionalidades en 5 minutos con la participación de todos, publicar y enviar el enlace a tiempo, y justificar la ausencia de modelos formales y de `PUT`/`PATCH`/`DELET`